



Ma Todo List

Rapport Technique — Application Cordova

HTML5 · CSS3 · JavaScript + jQuery · Swipe Gestures — ESP/UCAD 2026



UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR
École Supérieure Polytechnique — ESP

RAPPORT TECHNIQUE

Ma Todo List

Application Mobile — Apache Cordova

Documentation complète — HTML · CSS · JavaScript + jQuery

Swipe gestures, localStorage, filtres, modal bottom sheet — expliqués étape par étape

5 sections HTML
documentées

CSS vert thème sombre

17 fonctions JS
documentées

Swipe touch native
mobile

Auteur

Salif Biaye

Fondateur / CEO — DIC3 Informatique & Télécommunications

Application :	Ma Todo List
Technologie :	Apache Cordova · HTML5 · CSS3 · JavaScript + jQuery 3.7
Fonctionnalité clé :	Swipe gestures (touch events), localStorage, filtres
Module :	Développement Mobile — DIC3 Informatique & Télécommunications
Université :	UCAD — École Supérieure Polytechnique (ESP)
Auteur :	Salif Biaye — Édition Mai 2026

Sommaire

N°	Section	Contenu
1	Vue d'ensemble	Objectif, tech stack, gestion des tâches
2	Structure HTML	Header, filtres, liste, FAB, modal
3	Feuille de style CSS	Palette verte, swipe panels, animations
4	Logique JavaScript	17 fonctions : swipe, CRUD, modal, filtres
5	Captures d'écran	Liste, swipe, modal, état vide
6	Synthèse	Bilan technique et compétences

SECTION 1

Vue d'ensemble

Objectif, pile technologique et fonctionnement global de l'application

1.1 Qu'est-ce que Ma Todo List ?

Ma Todo List est une application mobile de gestion de tâches développée avec **Apache Cordova**. Elle permet d'ajouter, compléter et supprimer des tâches avec une interface inspirée des meilleures apps natives. La caractéristique principale est le système de **swipe (glissement tactile)** : glisser à droite pour valider une tâche, à gauche pour la supprimer.

Les tâches sont persistées localement via **localStorage** — elles survivent à la fermeture de l'application. L'interface propose des **filtres** (Toutes/Actives/Terminées) et un bouton flottant (FAB) pour ajouter de nouvelles tâches.

1.2 Pile technologique

Technologie	Rôle	Version
Apache Cordova	Wrapper natif Android/iOS	Dernière stable
HTML5	Structure : header, liste ul/li, modal, FAB	Standard W3C
CSS3	Thème vert sombre, swipe panels, animations	Standard W3C
JavaScript (IIFE)	Logique CRUD, filtres, rendu DOM	ES6+
jQuery 3.7.1	Sélecteurs DOM, events, helpers escape()	CDN code.jquery.com
Font Awesome 6.5	Icônes (check, trash, plus, xmark)	CDN Cloudflare
localStorage	Persistence des tâches (clé : 'todos')	Web API native

1.3 Flux utilisateur complet

- **Ajouter une tâche** : clic sur le FAB (+) → modal bottom sheet → saisie → Enregistrer.
- **Compléter une tâche** : swipe droit (≥60px) → flash vert → tâche barrée.
- **Supprimer une tâche** : swipe gauche → panneau rouge visible → clic ou swipe fort.
- **Filtrer** : boutons Toutes / Actives / Terminées → liste mise à jour instantanément.
- **Nettoyer** : bouton 'Supprimer terminées' visible si au moins 1 tâche complétée.
- **Persistence** : toute modification → localStorage → données conservées au rechargement.

Synthèse

- ❑ Fonctionnalité phare : swipe tactile natif (touchstart/touchmove/touchend)
- ❑ Persistance : localStorage JSON — survit à la fermeture de l'app
- ❑ jQuery utilisé pour la manipulation DOM et les event listeners
- ❑ Thème vert foncé (#071A0E → #4ADE80) avec mode clair automatique

SECTION 2

Structure HTML

Analyse complète des 5 sections du fichier `index.html`

2.1 Conteneur principal et en-tête

□ `index.html` — app + header

```
<div id="app">
  <header>
    <h1>Ma Todo List</h1>
    <p id="summary">0 tâche restante</p> <!-- Mis à jour par JS -->
  </header>

  <div id="filters">
    <button class="filter-btn active" data-filter="all">Toutes</button>
    <button class="filter-btn" data-filter="active">Actives</button>
    <button class="filter-btn" data-filter="completed">Terminées</button>
  </div>

  <ul id="task-list"></ul> <!-- Rempli dynamiquement par JS -->

  <div id="footer" style="display:none;">
    <button id="btn-clear-completed">Supprimer terminées</button>
  </div>
</div>
```

- **#summary** : compteur dynamique mis à jour par `renderSummary()`.
- **data-filter** : attribut lu par JS pour identifier le filtre actif.
- **#task-list (ul)** : liste vide au chargement, peuplée par `renderList()`.
- **#footer display:none** : caché par défaut, affiché uniquement si des tâches sont complétées.

2.2 Élément de tâche (généré dynamiquement)

Chaque tâche est générée par `buildTaskHtml()`. Le `<li>` contient 3 couches superposées : panneau gauche (done), panneau droit (delete), et le contenu principal par-dessus.

□ `index.js` — `buildTaskHtml` (HTML généré)

```
<li class="task-item" data-id="[timestamp]">
  <!-- Panneau gauche : swipe droit → cocher la tâche -->
  <div class="task-action-left">
```

```
<i class="fa-solid fa-check"></i>
</div>

<!-- Panneau droit : swipe gauche → supprimer -->
<div class="task-action-right">
  <i class="fa-solid fa-trash"></i>
</div>

<!-- Contenu principal (se translate via CSS transform) -->
<div class="task-content [done?]">
  <div class="task-check [checked?]"> ... </div>
  <span class="task-text">[texte de la tâche]</span>
</div>
</li>
```

2.3 Bouton FAB et modal d'ajout

□ index.html — FAB + modal

```
<!-- FAB : bouton flottant fixe en bas à droite -->
<button id="btn-add" title="Ajouter une tâche">
  <i class="fa-solid fa-plus"></i>
</button>

<!-- Modal bottom sheet -->
<div id="modal-overlay" class="hidden">
  <div id="modal">
    <div id="modal-header">
      <h2 id="modal-title">Nouvelle tâche</h2>
      <button id="modal-close"><i class="fa-solid fa-xmark"></i></button>
    </div>
    <div id="modal-body">
      <input type="text" id="f-task" placeholder="Ex : Faire les courses" maxlength="100">
      <p id="form-error" class="hidden">Veuillez saisir une tâche.</p>
    </div>
    <div id="modal-footer">
      <button id="btn-cancel">Annuler</button>
      <button id="btn-save">Enregistrer</button>
    </div>
  </div>
</div>
```

La classe `hidden` est ajoutée/retirée par jQuery pour afficher/masquer le modal. L'overlay sombre derrière le modal ferme celui-ci en cas de clic extérieur.

Synthèse

- ❑ 3 couches dans chaque li : panneau gauche / panneau droit / contenu (translateX)
- ❑ data-id : attribut clé reliant le DOM aux données en mémoire
- ❑ FAB fixed : reste visible pendant le scroll de la liste
- ❑ Modal overlay : fermeture en cliquant hors du panneau blanc

SECTION 3

Feuille de style CSS

Palette verte, système de swipe, animations et thème sombre

3.1 Palette de couleurs

Élément	Hex	Usage
fond global	#071A0E	Fond arrière très sombre (vert-noir)
header gradient	#14532D → #4ADE80	Dégradé en-tête du haut vers le bas
accent principal	#16A34A	Boutons, bordures actives, FAB
accent clair	#4ADE80	Hover, tâches complétées (lime)
texte	#E0F0E5	Texte principal sur fond sombre (vert clair)
panneau done	#16A34A	Panneau de gauche (swipe droit)
panneau delete	#C0392B	Panneau de droite (swipe gauche)
tâche barrée	#4ADE80 barré	Texte avec text-decoration: line-through

3.2 Mécanisme de swipe CSS

Le swipe est rendu possible par la superposition de 3 éléments dans chaque . Les panneaux d'action sont **positionnés en absolu** derrière le contenu principal. Le contenu se translate via translateX pour révéler le panneau sous-jacent.

❏ index.css – structure swipe

```
.task-item { position: relative; overflow: hidden; }

/* Panneaux cachés derrière le contenu */
.task-action-left, .task-action-right {
  position: absolute; top: 0; bottom: 0; width: 80px;
  display: flex; align-items: center; justify-content: center;
  opacity: 0; /* Transparents par défaut, opacity augmentée par JS */
}

.task-action-left { left: 0; background: #16a34a; } /* Vert – done */
.task-action-right { right: 0; background: #c0392b; } /* Rouge – delete */

/* Contenu principal positionné au-dessus */
.task-content {
  position: relative; z-index: 1;
```



```
/* translateX g  r   par JS via setTranslate() */
}
```

3.3 Animations CSS

Animation	Dur��e	D��clencheur	Effet
slideDown	0.2s ease	Ajout d'une t��che	La t��che entre par le bas (translateY)
slideOutLeft	0.28s ease	Suppression	La t��che sort vers la gauche
flashGreen	0.35s ease	Validation (done)	Flash de fond vert sur la t��che
slideUp	0.25s ease	Ouverture modal	Le modal monte depuis le bas
fadeIn	0.18s ease	Overlay modal	Fondu de l'arri��re-plan sombre

```
index.css - animation flash done

@keyframes flashGreen {
  0% { background-color: transparent; }
  30% { background-color: rgba(74, 222, 128, 0.35); }
  100% { background-color: transparent; }
}

.task-item.anim-done { animation: flashGreen 0.35s ease; }

/* JS : $item.addClass("anim-done") -> setTimeout(removeClass, 350) */
```

Synth  se

- Swipe CSS : 3 couches superpos  es — panneaux positionn  s absolute, contenu z-index:1
- Opacity progressive : JS augmente l'opacit   du panneau visible au fur et    mesure
- 5 animations CSS : slideIn, slideOutLeft, flashGreen, slideUp, fadeIn
- Mode clair : prefers-color-scheme media query — fond #f0faf4, texte #0d2016

SECTION 4

Logique JavaScript

IIFE Module avec jQuery — 17 fonctions documentées par groupe

4.1 Structure et état global

□ `index.js` — module + state

```
var App = (function ($) { // jQuery injecté en paramètre
  var STORAGE_KEY = "todos";

  var REVEAL_W = 80; // Largeur px du panneau swipe
  var SWIPE_COMMIT = 60; // Distance minimale pour valider un swipe
  var DIR_THRESHOLD = 6; // Distance avant de locker la direction

  var state = {
    tasks: load(), // Tableau des tâches depuis localStorage
    filter: "all", // Filtre actif : all | active | completed
    $open: null // Référence jQuery de l'item swipé ouvert
  };
  // ...
}($)); // jQuery passé immédiatement

$(App.init.bind(App)); // Démarrage après chargement DOM + jQuery
```

4.2 Stockage — load() et persist()

□ `index.js` — load / persist

```
function load() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]");
    // Retourne [] si clé absente ou JSON invalide
  } catch (_) { return []; } // Sécurité : JSON.parse peut lancer
}

function persist() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(state.tasks));
  // Sérialise le tableau complet — appelé après chaque modification
}
```

`escape(t) : $('<div>').text(t).html()` — astuce jQuery pour échapper les caractères HTML et prévenir les injections XSS.

plural(n) : construit la phrase '1 tâche restante' / '3 tâches restantes' avec accord grammatical français.

4.3 Filtrage — getFiltered()

□ index.js — getFiltered

```
function getFiltered() {
  return state.tasks.filter(function (t) {
    if (state.filter === "active") return !t.done; // Tâches non faites
    if (state.filter === "completed") return t.done; // Tâches faites
    return true; // "all" → tout retourner
  });
}
```

4.4 Rendu HTML — buildTaskHtml() et buildEmptyHtml()

buildTaskHtml(t) génère le HTML de chaque tâche avec ses 3 couches. buildEmptyHtml() génère un état vide contextuel selon le filtre actif.

□ index.js — buildEmptyHtml

```
function buildEmptyHtml() {
  var cfg = {
    completed: { icon: "fa-circle-check", msg: "Aucune tâche terminée" },
    active: { icon: "fa-hourglass-half", msg: "Aucune tâche active" },
    all: { icon: "fa-list-check", msg: "Ajoutez votre première tâche !" }
  };
  var c = cfg[state.filter];
  // Retourne un <li> avec l'icône et le message adaptés au filtre courant
}
```

4.5 Actions CRUD — addTask, toggleTask, deleteTask, clearCompleted

□ index.js — CRUD actions

```
// Ajouter : insère en tête du tableau (unshift = tâche récente en premier)
function addTask(text) {
  state.tasks.unshift({ id: Date.now(), text: text, done: false });
  persist(); render();
}

// Basculer done/undone
function toggleTask(id) {
```

```

var t = state.tasks.find(function (t) { return t.id === id; });
if (!t) return;
t.done = !t.done;
persist(); render();
}

// Supprimer avec animation sortie
function deleteTask(id, $item) {
$item.addClass("anim-delete"); // Déclenche slideOutLeft 0.28s
setTimeout(function () {
state.tasks = state.tasks.filter(function (t) { return t.id !== id; });
persist(); render();
}, 280); // Attendre la fin de l'animation
}

// Nettoyer : garder uniquement les tâches non terminées
function clearCompleted() {
state.tasks = state.tasks.filter(function (t) { return !t.done; });
persist(); render();
}

```

4.6 Swipe — setTranslate, openItem, closeOpen

□ index.js — swipe helpers

```

// Translate le contenu et ajuste l'opacité du panneau révéle
function setTranslate($item, x, animate) {
$item.find(".task-content").css({
transition: animate ? "transform 0.22s ease" : "none",
transform: x ? "translateX(" + x + "px)" : ""
});
// x>0 : panneau gauche (done) visible
// x<0 : panneau droit (delete) visible
if (x > 0) {
var pct = Math.min(x / REVEAL_W, 1); // 0 → 1
$item.find(".task-action-left").css("opacity", pct);
$item.find(".task-action-right").css("opacity", 0);
} else if (x < 0) { /* symétrique pour panneau droit */ }
else { /* reset opacity à 0 */ }
}

function openItem($item) {
if (state.$open && state.$open[0] !== $item[0]) closeOpen(true);

```

```

state.$open = $item;
$item.addClass("revealed");

setTranslate($item, -REVEAL_W, true); // Glisse de -80px
}

function closeOpen(animate) {
if (!state.$open) return;

setTranslate(state.$open, 0, animate);

state.$open.removeClass("revealed");

state.$open = null;
}

```

4.7 Swipe — bindSwipe() (gestionnaire tactile)

La fonction `bindSwipe()` attache 3 gestionnaires tactiles sur `#task-list` en utilisant la délégation d'événements jQuery. Un objet touch local stocke l'état du geste en cours.

□ index.js — bindSwipe logic

```

function bindSwipe() {
var touch = { x: 0, y: 0, dir: null, active: false };

// touchstart : mémoriser le point de départ
$("#task-list").on("touchstart", ".task-item", function (e) {
var t = e.originalEvent.touches[0];
touch.x = t.clientX; touch.y = t.clientY;
touch.dir = null; touch.active = true;
});

// touchmove : calculer déplacement et animer
$("#task-list").on("touchmove", ".task-item", function (e) {
var dx = t.clientX - touch.x, dy = t.clientY - touch.y;
// Locker direction après DIR_THRESHOLD (6px)
if (!touch.dir && Math.max(Math.abs(dx), Math.abs(dy)) > DIR_THRESHOLD)
touch.dir = Math.abs(dx) > Math.abs(dy) ? "h" : "v";
if (touch.dir !== "h") return; // Scroll vertical → ignorer
e.preventDefault(); // Bloquer le scroll si swipe horizontal
setTranslate($(this), clamped, false);
});

// touchend : décider de l'action selon distance parcourue
// dx < -60 → openItem (panneau delete)
// dx > +60 → toggleTask avec flash vert
// sinon → fermer (snap back)

```

}

4.8 Modal — openModal, closeModal, saveTask

□ index.js — modal

```
function openModal() {
  $("#f-task").val("");
  $("#form-error").addClass("hidden");
  $("#modal-overlay").removeClass("hidden"); // Afficher le modal
  setTimeout(function () { $("#f-task").focus(); }, 150); // Focus clavier
}

function closeModal() {
  $("#modal-overlay").addClass("hidden"); // Masquer sans réinitialiser
}

function saveTask() {
  var text = $("#f-task").val().trim();
  if (!text) { $("#form-error").removeClass("hidden"); return; }
  addTask(text); // Appelle persist() + render()
  closeModal();
}
```

Synthèse

- Module IIFE + jQuery : scope privé, jQuery injecté proprement en paramètre
- State objet centralisé : tasks[], filter, \$open — source unique de vérité
- Swipe 3 phases : touchstart (mémoriser) / touchmove (animer) / touchend (décider)
- deleteTask : animation 280ms avant suppression réelle — UX fluide
- 17 fonctions : load, persist, escape, plural, getFiltered, buildTaskHtml, buildEmptyHtml, renderList, renderSummary, renderFooter, render, addTask, toggleTask, deleteTask, clearCompleted, setFilter, setTranslate, openItem, closeOpen, bindSwipe, openModal, closeModal, saveTask, bindEvents, init

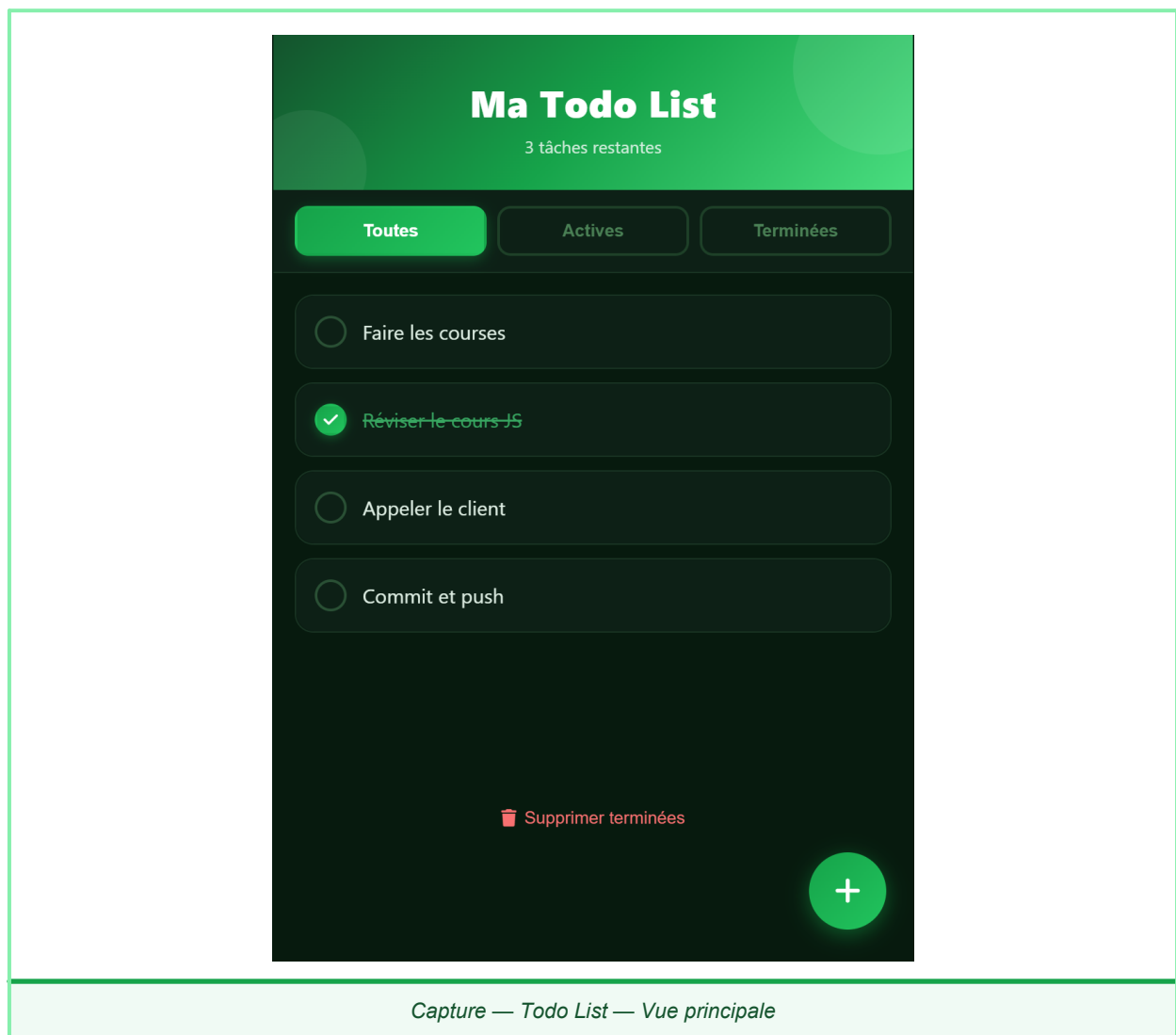
SECTION 5

Captures d'écran

Placer les images dans : C:\Users\DELL\Downloads\medy\screens

5.1 Liste des tâches

Vue principale avec les tâches, les filtres et le FAB vert.



5.2 Swipe en action

Panneau rouge visible après swipe gauche, icône poubelle révélée.

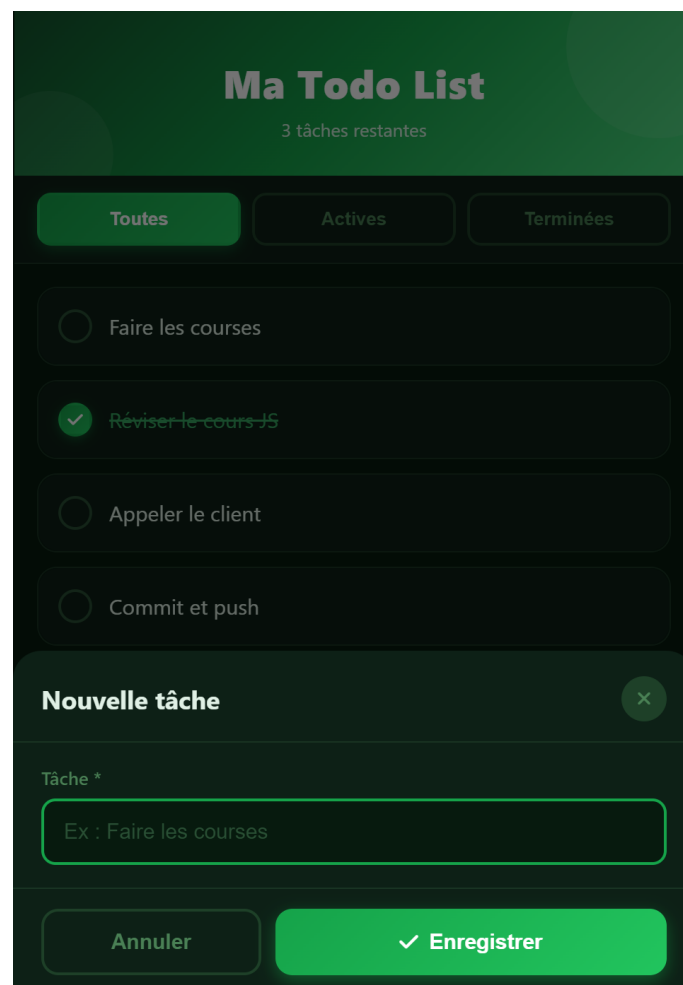


Todo List — Swipe gauche (delete)

Placer la capture ici : C:\Users\DELL\Downloads\medy\screens\todo_swipe.png

5.3 Modal d'ajout

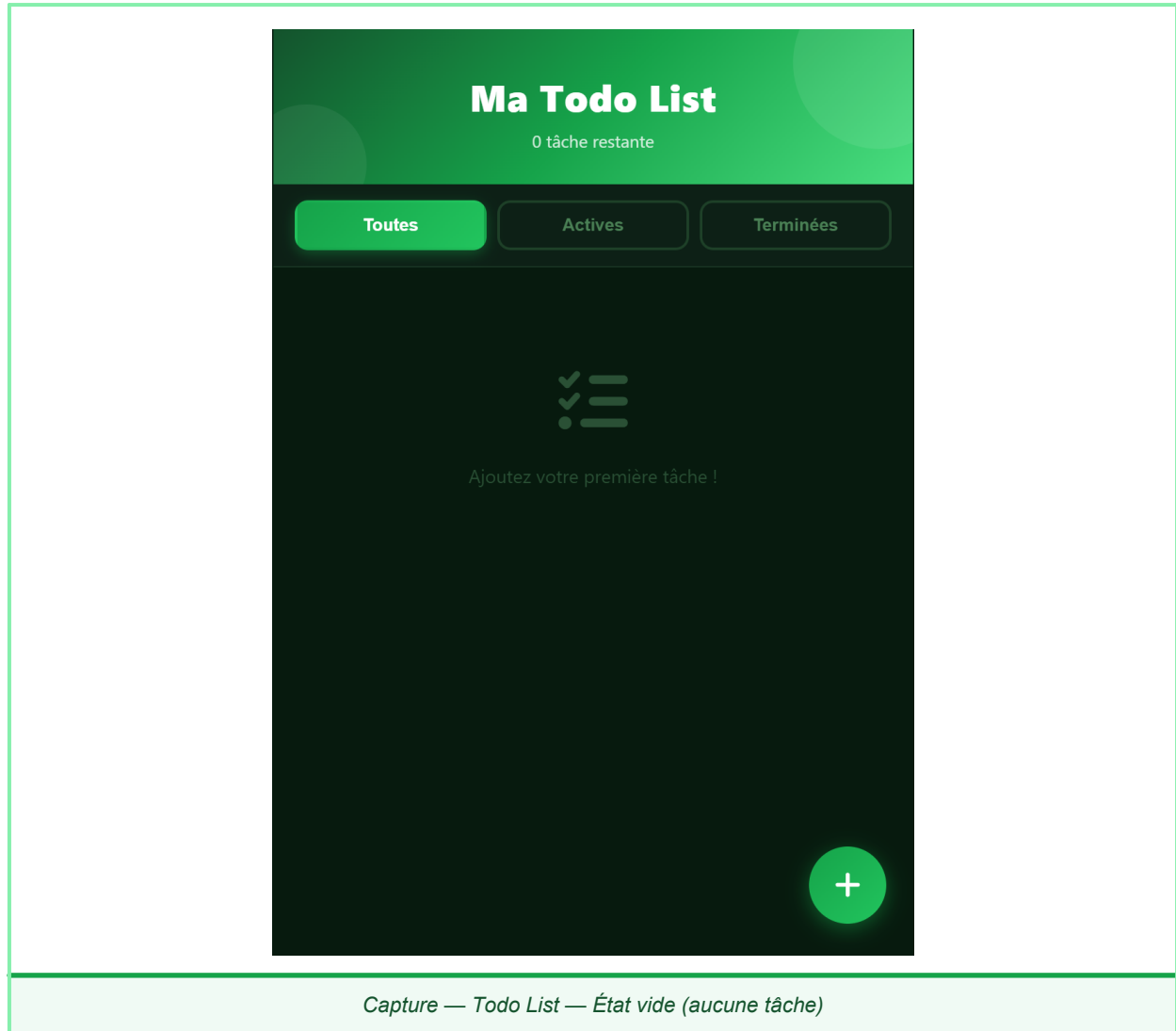
Bottom sheet avec champ de saisie et boutons Annuler / Enregistrer.



Capture — Todo List — Modal ajout tâche

5.4 État vide

Message adapté au filtre actif si aucune tâche ne correspond.



SECTION 6

Synthèse finale

Bilan technique et compétences démontrées

6.1 Tableau récapitulatif

Aspect	Détail	Niveau
Swipe tactile	Touch events natifs — lock de direction, snap, seuils	☐ Avancé
Architecture JS	IIFE + jQuery, state centralisé, fonctions atomiques	☐ Solide
Persistance	localStorage JSON — load/persist après chaque action	☐ Complet
Animations	5 animations CSS : slide, flash, fadeIn — timing précis	☐ Soigné
UX mobile	FAB, modal bottom sheet, feedback visuel, état vide	☐ Optimisé
Sécurité	escape() XSS via jQuery, pas d'innerHTML direct	☐ Bon

Synthèse

- ☐ Swipe natif : lock direction h/v, seuil 60px, snap animé — implémentation complète
- ☐ 3 couches HTML : panneaux abs + content translateX — sans bibliothèque swipe
- ☐ CRUD complet : add (unshift), toggle, delete (animé), clearCompleted
- ☐ 17+ fonctions organisées en 4 groupes : storage, render, actions, modal

Ma Todo List

Rapport Technique — Application Cordova

UCAD — École Supérieure Polytechnique (ESP)

Salif Biaye — Fondateur / CEO

Module : Développement Mobile — DIC3 — Édition Mai 2026

Document produit dans le cadre du cursus DIC3 — Usage académique uniquement